



La shell

Di cosa parleremo:

01 UNIX e la sua filosofia

03 BASH

05 Muoversi nel filesystem

07 Input, output e redirect

09 Script e permessi

02 Cosa è la shell

04 BASH prompting

06 Creare, spostare, cancellare

08 Pipes: combinare i comandi

01

UNIX e la sua filosofia

Da dove viene la riga di comando, e l'idea che la rende potente.

UNIX, 1969 • Bell Labs

La filosofia UNIX in una frase

Costruisci tanti **piccoli strumenti**,
ognuno dei quali fa **bene una cosa sola**, ma che possono
essere **combinati** per ottenere cose più potenti.
Idea: l'output di un programma diventa l'input di un altro.

Un'idea semplice e potente: in UNIX **tutto è un file**, perfino
l'hardware, e la maggior parte è testo. È questa uniformità che fa
combinare gli strumenti.

Kernel, shell, sistema operativo

kernel

astrae sull'hardware: lo stesso software gira su computer diversi

shell

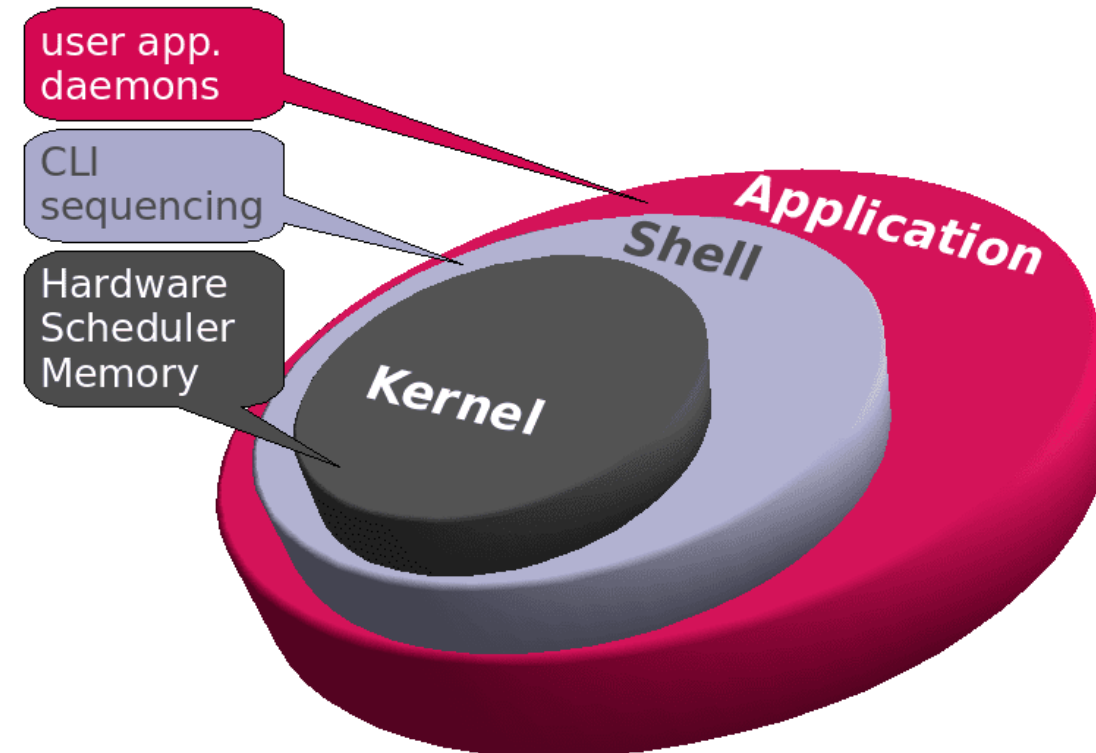
lo strato più esterno: permette all'utente di lanciare i programmi

sistema operativo

l'insieme di kernel, programmi di sistema e shell

Shell

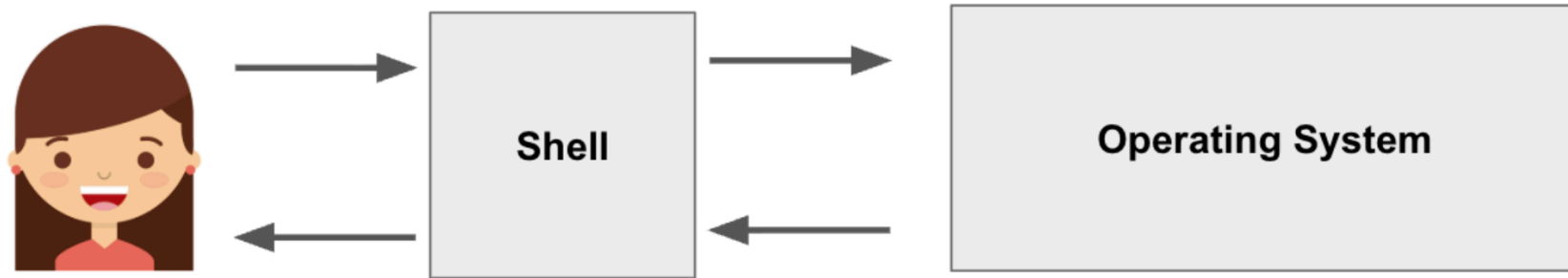
```
gfg0913@administrator-Latitude-3420: ~/Desktop/Personal
SahilKamali:~/Desktop/GFG$ ./script.sh
Current Directory:
/home/gfg0913/Desktop/GFG
Files in Directory:
total 8
drwxrwxr-x 2 gfg0913 gfg0913 4096 Feb 18 18:07 completedWroks
-rwxrwxr-x 1 gfg0913 gfg0913 143 Feb 18 18:08 script.sh
SahilKamali:~/Desktop/GFG$
```



02

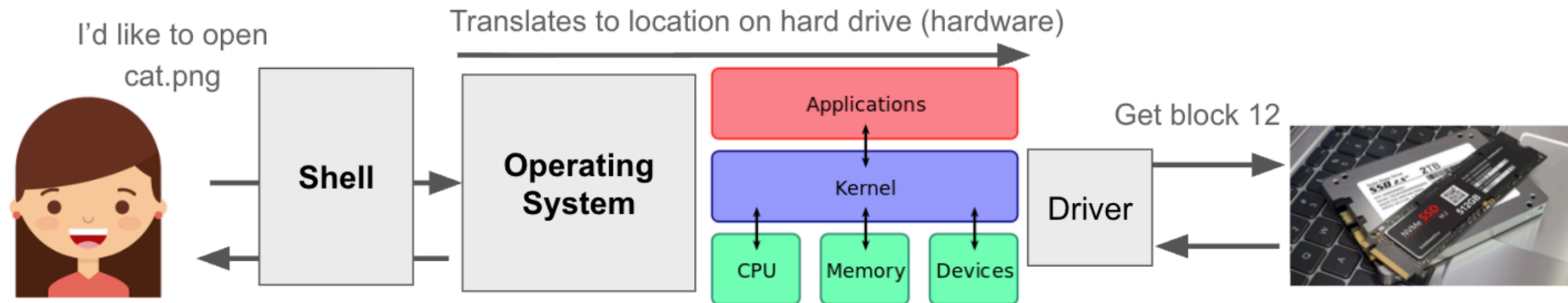
Cosa è la shell

In informatica, una shell è un programma che mette a disposizione dell'utente o di altri programmi i servizi del sistema operativo.



Perchè si chiama shell? Perchè è lo strato più esterno del sistema operativo

Se vuoi fare qualsiasi cosa con il tuo computer, stai interagendo con il sistema operativo, che tu ne sia consapevole o meno.



Il sistema operativo gestisce le risorse software e hardware del tuo computer. Pianifica il lavoro sulle CPU, assegna memoria ai processi e gestisce il file system e i dispositivi di rete.

GUI contro CLI

Due modi di comandare il computer

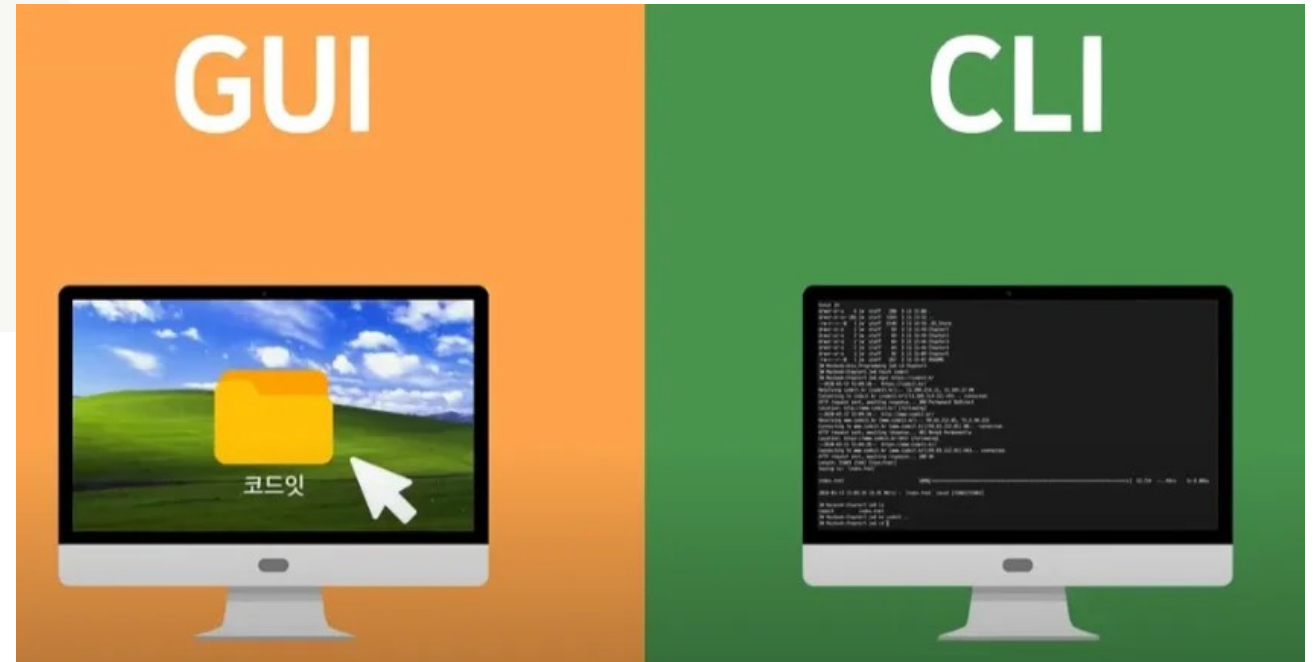
Interfaccia grafica (GUI)

Clic, trascinamenti, menu. Intuitiva da imparare, ma **scalamale** sui compiti ripetitivi.

Riga di comando (CLI)

Si legge e si scrive testo. Perfetta per rendere **automatici e velocissimi** i compiti ripetitivi.

Copiare la terza riga di **mille** file in un unico file: col mouse ci passi ore (e sbagli); con la shell, una breve sequenza lo fa identica e quasi istantanea.



Perché la shell (CLI)?

- Permette di lavorare in modo efficiente con la tastiera invece che con il mouse
- È possibile automatizzare facilmente i compiti scrivendo script
- Alcune macchine, come i server e i sistemi embedded, ti offrono solo una shell a riga di comando con cui lavorare

03

Bash

Bourne Again Shell

Bash · Perché impararlo

Cinque buoni motivi

Automazione: trasformare una sequenza di comandi in uno **script**.

Composizione: collegare programmi diversi con le **pipe** (**|**).

Controllo del sistema: file, processi, permessi, variabili d'ambiente...

Semplicità: molte operazioni richiedono una sola riga.

Portabilità: fondamentale in Linux/Unix, e molto usato su server e cloud.

L'idea chiave dietro Bash

Far lavorare insieme ciò che hai già

Bash non serve tanto a "costruire applicazioni", quanto a far lavorare insieme il sistema operativo e i programmi che abbiamo già.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The title bar of the window says "bash". The command prompt is a green "\$" followed by the command "grep 'ERROR' *.log | sort > errori.txt" in white text.

```
bash  
$ grep "ERROR" *.log | sort > errori.txt
```

Cerca gli errori nei log → li ordina → salva il risultato in un file. Tre strumenti, una riga.

Bash vs PowerShell

Due filosofie a confronto

	 Bash	 PowerShell
Ecosistema principale	Linux / Unix	Windows
Filosofia	Comandi + testo	Comandi + oggetti
Script	.sh	.ps1
Output nella pipeline	Testo	Oggetti .NET
File system / processi	★★★★★	★★★★★
Amministrazione Windows	★★★★☆☆	★★★★★
Linux / server / cloud	★★★★★	★★★★☆☆

Bash → passa testo tra i programmi

```
bash  
  
$ ls | grep ".txt"
```

BASH

Filosofia Unix: piccoli programmi che fanno bene una cosa e che si possono combinare.

PowerShell → passa oggetti tra i comandi

```
powershell  
  
> Get-ChildItem | Where-Object Extension -eq  
".txt"
```

POWERSHELL

Progettata per lavorare con gli oggetti del sistema e dell'ecosistema Windows/.NET.

Come ottenere Bash su PC Windows

LA PIÙ SEMPLICE

Git Bash

Arriva con Git: una vera bash, copre quasi tutti i comandi.

LINUX VERO

WSL

`wsl --install`. Non è una VM tradizionale; serve per le poche cose che a Git Bash mancano.

ZERO INSTALL

Browser

Un terminale Linux nel browser (cmdchallenge, replit): va su qualsiasi macchina.



<https://gitforwindows.org>

Guida installazione:

<https://www.geeksforgeeks.org/git/working-on-git-bash/>

04

Bash prompting

Ogni parte dice qualcosa

[**studente**@**precorso** ~]\$

studente = utente · **precorso** = nome macchina · ~ = cartella corrente

Il simbolo finale (\$) **non si digita mai**: indica solo dov'è il prompt.

Tu scrivi il comando dopo, e premi **Invio**.

La tastiera è il tuo alleato

TAB completa nomi

↑ ↓ comandi già dati

CTRL-R cerca nella history

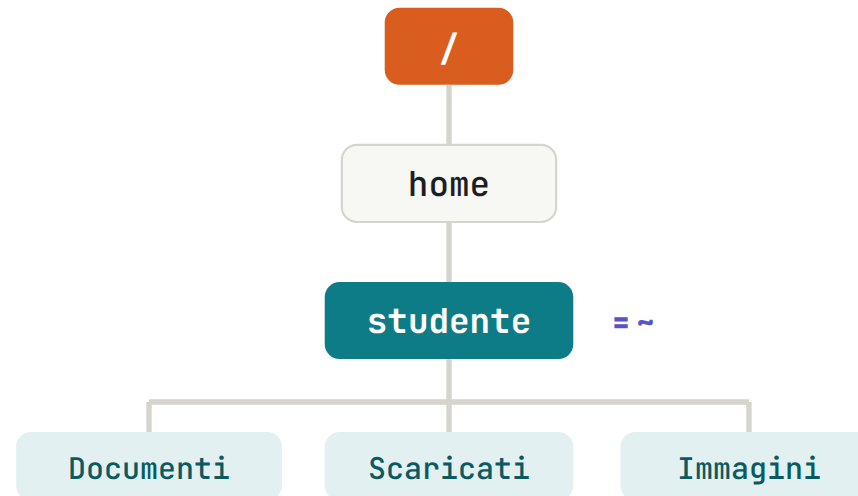
CTRL-C interrompe

CTRL-D fine input

05

Muoversi nel file system

In cima la radice, poi si dirama tutto



Nomi sensibili alle maiuscole: `File.txt` $\neq$ `file.txt`.

Scorciatoie di directory

~ la **home**, la tua cartella personale

/ la **radice** (root), la più in alto

. la **cartella corrente**

.. la **cartella superiore** (il genitore)

Comandi Bash Filesystem

pwd · ls · cd

```
bash

[studente@precorso ~]$ pwd           # dove sono?
/home/studente

[studente@precorso ~]$ ls           # elenca i file
Documenti  Scaricati  Immagini

[studente@precorso ~]$ cd Documenti # entra nella cartella
[studente@precorso Documenti]$ cd .. # torna su
```

06

Creare, spostare, cancellare

05 · I comandi essenziali

mkdir · touch · mv · cp · cat

```
bash

$ mkdir prove                # crea la cartella
$ touch test.txt             # crea un file vuoto
$ mv test.txt appunti.txt    # sposta / rinomina
$ cp appunti.txt copia.txt   # copia (cp -r per cartelle)
$ cat appunti.txt            # mostra il contenuto
```

La wildcard `*` sostituisce qualsiasi sequenza: `m*e` → mine, myfile, male...

La brace expansion e il comando rm

Tanti file in un colpo

La shell espande le parentesi *prima* di eseguire:

```
touch appunti{1..15}.txt
```

 crea 15 file;

```
mkdir dir{1..15}
```

 15 cartelle;

```
touch {a,b,c}.txt
```

 i tre elencati.

rm: cancella per sempre

rm rimuove un file, `rm -r` un'intera cartella.

Non esiste un cestino: è immediato e irreversibile.

Prima di premere Invio, rileggi cosa stai cancellando.

Esercizio:

1. Navigazione e creazione della cartella:
 - Usa i comandi che conosci per visualizzare in quale directory ti trovi attualmente.
 - Spostati sul Desktop (se non sei già lì).
 - Crea una nuova cartella chiamata ProgettoTerminale.
2. Navigazione e gestione dei file:
 - Entra nella cartella ProgettoTerminale.
 - All'interno, crea un'altra cartella chiamata Documenti.
 - Crea tre file vuoti nella cartella Documenti con i nomi appunti.txt, compiti.txt e bozza.txt.
3. Navigazione e manipolazione con caratteri speciali:
 - Torna indietro nella cartella principale ProgettoTerminale senza utilizzare il percorso assoluto.
 - Crea una nuova cartella chiamata Immagini.
 - Crea due file vuoti con i nomi foto1.png e foto2.png nella cartella Immagini usando caratteri speciali per evitare di digitare l'intero nome del file due volte.
4. Lista e verifica:
 - Visualizza la lista completa dei file all'interno della cartella ProgettoTerminale e delle sue sottocartelle, assicurandoti che tutto sia stato creato correttamente.
 - Usa un carattere speciale per visualizzare solo i file .txt presenti in Documenti.



Riassunto comandi

- pwd → print working directory
- ls → list
- mkdir → make directory
- touch → create empty file / update timestamp
- cd → change directory
- . → current directory
- .. → parent directory
- ~ → home directory

I comandi cp, mv, rm possono anche lavorare su più file o cartelle usando caratteri speciali *

Esempio1: se vogliamo copiare tutti i file .txt in una cartella

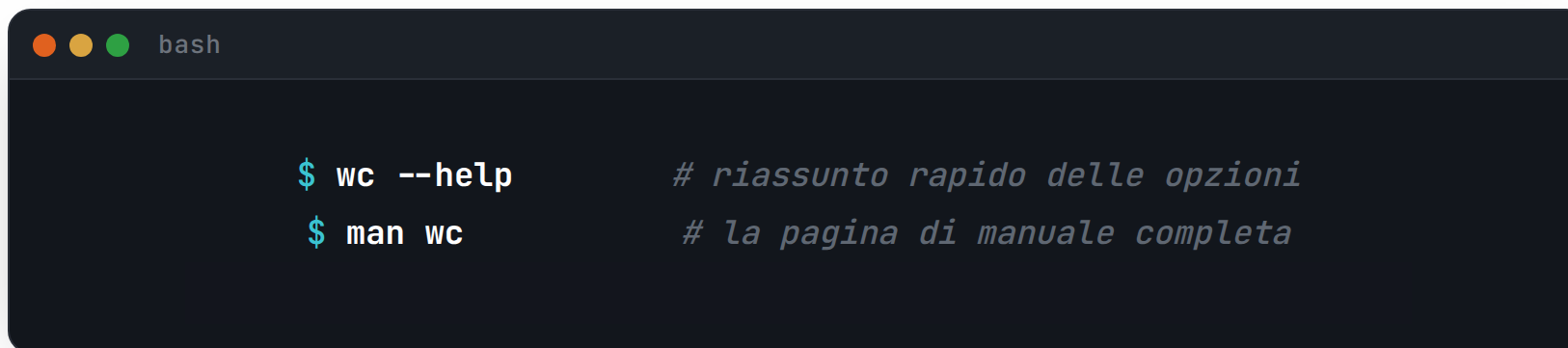
```
cp *.txt Sottocartella1/
```

Esempio2: vogliamo eliminare tutte le estensioni png

```
rm *.png
```

La shell documenta sé stessa

Nessuno ricorda tutte le opzioni

A terminal window with a dark background and a title bar containing three colored circles (orange, yellow, green) and the text 'bash'. The terminal shows two commands and their descriptions in a light blue monospace font.

```
$ wc --help      # riassunto rapido delle opzioni
$ man wc         # la pagina di manuale completa
```

Per uscire da *man* basta premere *q*

Esercizio:

- Vai nella tua home directory
- Crea una cartella chiamata `Esercizio` e dentro una sottocartella `Test`
- Crea 3 file vuoti: `prova1.txt`, `prova2.txt`, `prova3.txt`
- Copia tutti i file `*.txt` dentro `Test`
- Sposta `prova1.txt` dalla cartella corrente alla tua home directory
- Rinomina `prova2.txt` in `esempio.txt`
- Cancella `prova3.txt`
- Vai dentro `Test` ed esplora con `man rm` le opzioni disponibili
- Torna in `Esercizio` ed elimina la cartella `Test`



Riassunto comandi

- `pwd` → print working directory
- `ls` → list
- `mkdir` → make directory
- `touch` → create empty file / update timestamp
- `cd` → change directory
- `.` → current directory
- `..` → parent directory
- `~` → home directory



Riassunto comandi

- `man` → manual
- `cp` → copy
- `mv` → move
- `rm` → remove

07

Input, output e redirect

Se "tutto è un file", anche ciò che entra ed esce da un comando lo è.

I tre canali standard

Ogni comando legge e scrive su tre canali

standard input • stdin

il file da cui il programma legge

standard output • stdout

il file su cui scrive i risultati

standard error • stderr

il file su cui scrive gli errori

Di default sono collegati al terminale, ma la shell li può **redirigere**
(i.e. re-indirizzare).

Redirect verso e da un file

> sovrascrive · >> aggiunge · < legge

```
bash

$ echo "ciao" > saluto.txt      # scrive nel file (sovrascrive)
$ echo "riga 2" >> saluto.txt   # aggiunge in coda
$ cat < saluto.txt              # legge l'input dal file
                                ciao
                                riga 2
```

Comandi utili per i dati

grep	cerca righe che contengono un testo	sort	ordina le righe (-r ordine inverso -n ordine numerico)
sed	cerca-e-sostituisci	uniq	rimuove i duplicati
cut	estrae una "colonna"	wc	conta righe/parole
head/tail	prime / ultime righe	xargs	output → argomenti

head Mostra solo le prime righe di un file (di default 10)

```
head note.txt
```

```
head -n 5 note.txt
```

tail Mostra solo le ultime righe di un file (di default 10)

```
tail note.txt
```

```
tail -n 5 note.txt
```

wc Conta linee, parole e byte

Opzioni principali:

- -l → linee
- -w → parole
- -c → byte

```
1.wc dati_estesi.txt
```

```
# Linee, parole, byte
```

```
2.wc -l dati_estesi.txt
```

```
# Numero di linee
```

```
3.wc -w dati_estesi.txt
```

```
# Numero di parole
```

grep Cerca righe che contengono un pattern

Opzioni utili:

- `-v` → mostra le righe che non contengono il pattern

1. `grep "Trieste" dati_estesi.txt` # Tutti quelli che vivono a Trieste
2. `grep -v "Torino" dati_estesi.txt` # Tutti tranne quelli di Torino
3. `grep "^A" dati_estesi.txt` # Tutti i nomi che iniziano con A

Esercizio:

1. Conta quante righe ci sono nel file.
2. Conta quante parole ci sono in tutto il file.
3. Visualizza le prime 3 righe e salvale in un nuovo file chiamato `prime3.txt`.
4. Visualizza le ultime 2 righe e salvale in un nuovo file `ultime2.txt`.
5. Ordina alfabeticamente tutte le righe e salvale in `ordinato.txt`.
6. Ordina le righe in ordine inverso e visualizza solo a schermo.
7. Estrai solo la prima parola di ogni riga.
8. Cerca tutte le righe che contengono la parola shell.
9. Cerca tutte le righe che non contengono la parola Linux.

08

Le pipe: combinare comandi

L'output di uno diventa l'input del prossimo.

Il carattere | collega stdout a stdin



Nessun file temporaneo di mezzo: sono **parallele** e **pigre** (leggono solo i dati che servono).

Pochi comandi, operazioni potenti

bash

```
$ ls ~/Documenti | grep -i relazione    # solo i file "relazione"  
$ env | wc -l                          # quante variabili d'ambiente?  
  
$ wc -l * | sort -n | head -n 5        # i 5 file con meno righe
```

Variabili di shell e d'ambiente

Variabili di shell

Interne, in minuscolo: `var=valore`.

Variabili d'ambiente

Passate ai programmi, in MAIUSCOLO: `export VAR=valore`.

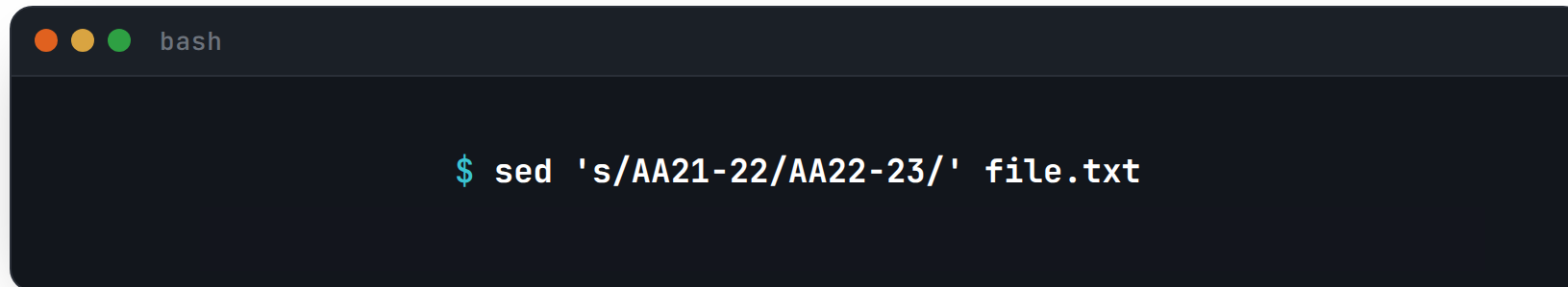
```
bash

$ nome="Ada"
$ echo $nome           # $ usa il valore
Ada
$ echo $PATH           # dove cerca i comandi
/usr/local/bin:/usr/bin:/bin
```

s/CERCA/SOSTITUISCI/

s / AA21-22 / AA22-23 /

s = sostituzione · testo da cercare · testo con cui sostituire

A terminal window with a dark background and three colored window control buttons (orange, yellow, green) in the top-left corner. The text 'bash' is visible next to the buttons. The command '\$ sed 's/AA21-22/AA22-23/' file.txt' is entered in the terminal.

```
bash  
  
$ sed 's/AA21-22/AA22-23/' file.txt
```

09

Script e permessi

Salvare i comandi in un file, e controllare chi può usarlo.

Permessi:

chmod Modifica permessi dei file e delle cartelle

```
chmod [chi] [operatore] [permessi] file
```

- Chi: u (user/proprietario), g (gruppo), o (altri), a (tutti)
- Operatore: + (aggiungi), - (rimuovi), = (imposta esattamente)
- Permessi: r (read), w (write), x (execute)
- Esempi
 - `chmod u+x script.sh` # aggiunge permesso di esecuzione al proprietario
 - `chmod g-w file.txt` # rimuove permesso di scrittura al gruppo
 - `chmod a=r file.txt` # tutti possono solo leggere
- Sintassi Ottale
 - Ogni permesso ha un valore numerico: r = 4, w = 2, x = 1
 - Si sommano i valori: 7 = rwx, 6 = rw-, 5 = r-x, 4 = r—
 - `chmod 755 script.sh` -> proprietario = 7 (rwx), gruppo = 5 (r-x), altri = 5 (r-x)

```
-rwxr-xr--
```

ls -l Mostra i permessi dei file nella directory corrente

Esercizio:

1. Vai nella tua home e crea una nuova cartella `permessi`
2. Crea un file `segreto.txt` e scrivici dentro un messaggio
3. Verifica i permessi iniziali del file
4. Rimuovi a tutti il permesso di lettura
5. Prova a leggerlo con `cat`
6. Ridai al proprietario il permesso di lettura
7. Controlla di nuovo i permessi e verifica che adesso tu possa leggere il file

Script: una sequenza di comandi in un file

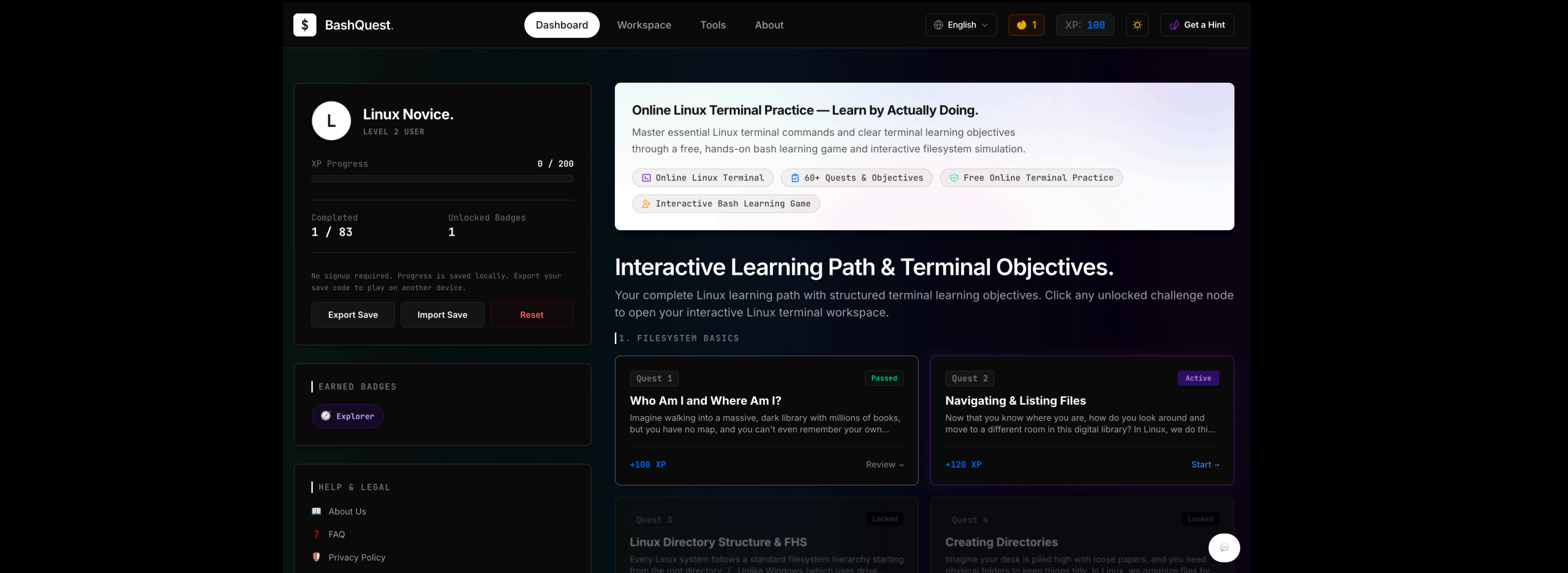
```
saluta.sh

#!/usr/bin/env bash # lo shebang
nome=$1             # 1° parametro
echo "Ciao, $nome!"
```

```
bash

$ chmod a+x saluta.sh # permesso
                        di eseg.
$ ./saluta.sh Ada
Ciao, Ada!
```

Parametri \$1 \$2 ..., output di un comando in `var=$(comando)`,
stato di uscita in `$?` (0 = ok).



<https://www.bashquest.com>



Command Challenge



View Solutions



Your first challenge is to print "hello world" on the terminal in a single command.

Hint: There are many ways to print text on the command line, one way is with the 'echo' command. Try it below and good luck!

(0)> █